First assignment Distributed Computing

Francesco Matano (S5253162), Davide Giusto (S5252480)

December 13, 2024

Contents

1 Introduction **2**
1.1 Delivery 2

2 What questions lead you to this simulator extension? **2**
2.1 Why do you think they are relevant? 2

3 How did you do that? **3**
3.1 Dynamic-D 3
3.1.1 How do we choose D? 3
3.2 Job Stealing 5

4 What is the answer to the original questions you asked? **5**

5 Why is that? What is your interpretation of the results you obtained? **10**

1 Introduction

1.1 Delivery

For the first assignment we must answer to these questions:

- What questions lead you to this simulator extension?
  - Why do you think they are relevant?
- How did you do that?
  - Try to be at the right level of detail: don't include obvious/boring details, and discuss anything that is not obvious.
  - Aim at a document that should be clear to somebody who knows both the original simulator and the programming language.
  - Describe any unexpected, surprising, and interesting things you found.
- What is the answer to the original questions you asked?
  - Why is that? What is your interpretation of the results you obtained?

2 What questions lead you to this simulator extension?

The question that led us to do this simulation extension was: "If we add to our simulator the functionalities of a dynamic-d and job stealing, can we obtain a new system that better performs than the original one?".

2.1 Why do you think they are relevant?

We believe that this question could be relevant because especially when the workload is high(lambda = 0.99), it is frequent that the queues are unbalanced and so, in this case, can be useful assign a larger value to D to increase the probability to choose the least loaded queue in the system; while if we have a situation where queues have almost the same size, it is useless to have a great D.

In addition the use of a dynamic D can bring improvements for computational overhead, because if we have a constant high D value when the queues have the same length it can be more expensive rather than having a dynamic D whose value decreases as the decrease of unbalancing between the queues.

We also think that we can distribute better the jobs inside the system, introducing a job stealing algorithm: because can be useful to "steal" jobs from an over-loaded queue and assign them to an under-loaded one.

3 How did you do that?

3.1 Dynamic-D

For the our purposes we ask ourself how to setup in a good way the dynamic value to D.
For do that we chose to use an unbalance system measure, the CV metric:

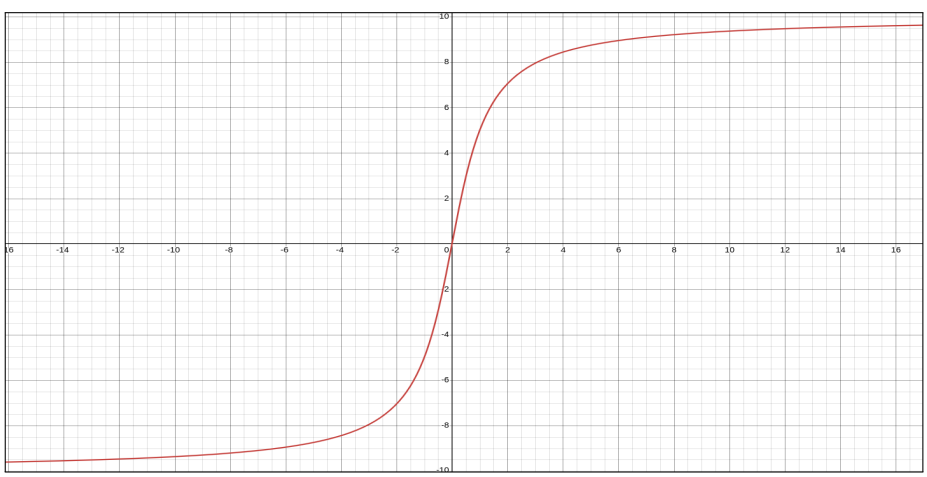
$$\mathbf{cv} = \frac{StandardDeviation}{AverageLoadQueus}.$$

If this coefficient value is higher than 1, this means that the jobs in the system are bad distributed and we need to sample an higher number of server.
Indeed, if this value is lower than 1 it means that the jobs in the system are not so bad distributed and we can select a less number of server.

3.1.1 How do we choose D?

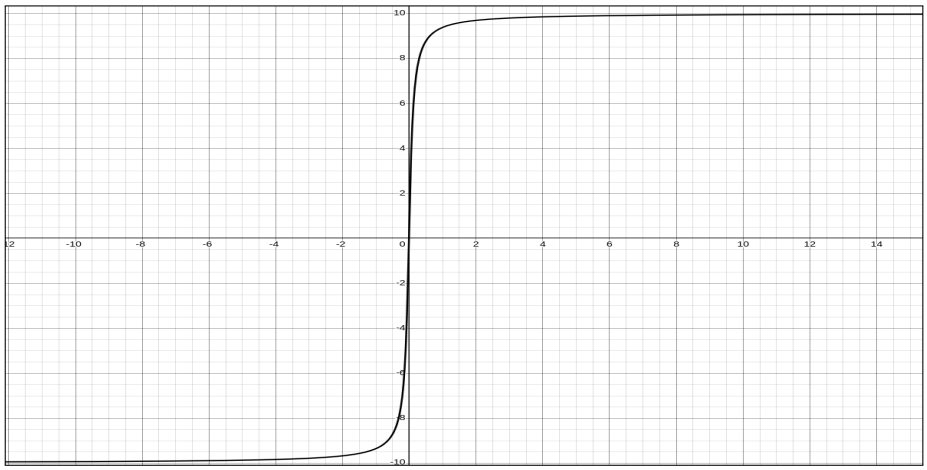
For the D choosing phase we have used different methods, taking into account that max_d is the 80 % of the total servers:

1. $dynamic_d = \frac{arctan(cv)*max_d*2}{\pi}$



In the first method we use this function that grows moderately as x (CV) increases.
We have noticed that CV value is almost always around 0.5: in this case the number of server that we assign will almost always the 30 % of 80 % of the servers that are in the simulation.
This means that in the most case we select the 24 % of the servers.

2. $dynamic_d = \frac{arctan(cv*max_d)*max_d*2}{\pi}$



For the second function we use a similar function, but it grows more faster than the first one.
When we use this function we obtain more frequently high D value, which is fine for an efficiency argument but not for an efficiency one.

3. $$\begin{cases} \frac{arctan(cv)*max_d*2}{\pi}, & \text{if CV <1 or queue length mean <3} \\ \frac{arctan(cv*max_d)*max_d*2}{\pi}, & \text{if CV >= 1} \end{cases} \tag{1}$$

For the third method, we decided to combine the previous two functions to take the advantages of them.
To do so, we decided to use the first function if the CV are less than 1 or if the average queues length is not so high, this because the CV is often high when the average queues length is small, but in this case we do not have to assign a large value to D.

Indeed, if both CV and average queue length are high, we need to sample a larger queue range, because we are in a situation of unbalanced queues.

4. For the fourth algorithm we think in an easier way: we based on the CV and average queue length values and so we regulate the D value:

   (a) **If CV <0.5 or if CV >1 and the tail length average is <= 1**:
   The D value will be equal to at the 20 % of the number of servers in the systems beacuse we are in an underload situation.

(b) **If CV <1**:
D will be equal to half of the servers in the system because we are neither in an overload situation nor in a underloaded one

(c) **None of the precedence points are true**:
The D value will be equal to 70 % of the servers that are present in the system because we are in an overload situation.

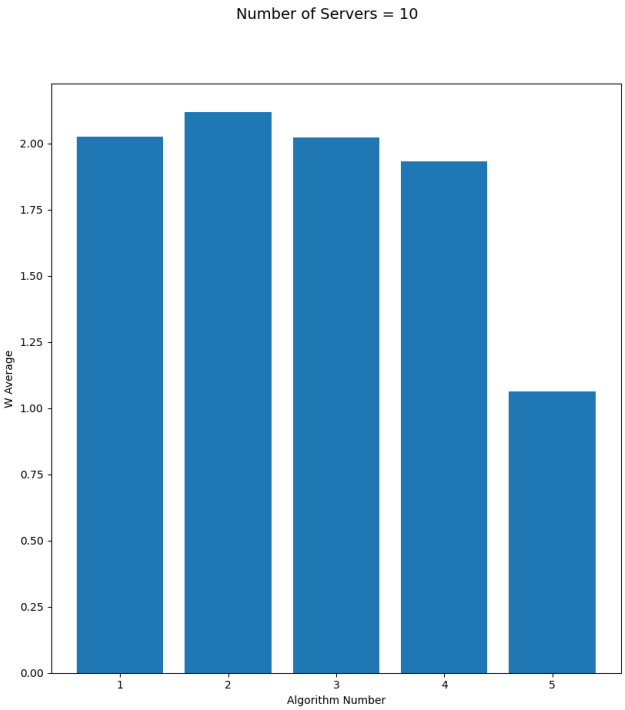
5. The fifth and final algorithm is the easiest.
The algorithm consists only of assigning to D a value equal to half of the total queues and it can be seen as the original system .
This algorithm don't use job stealing and not change the value of D during the entire simulation is it used just for the interpretation of the results .

3.2 Job Stealing

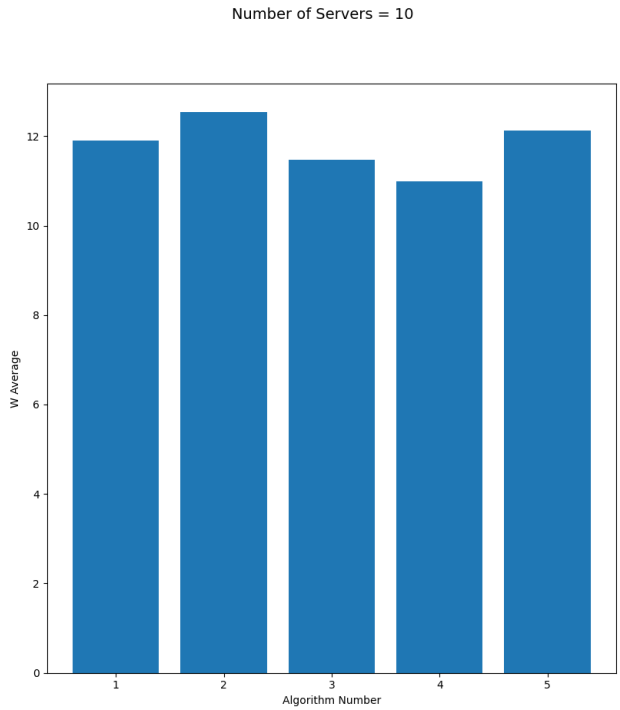
The job stealing is applied when a job is completed and the queue handling it is empty, when this happens we take the most loaded queue within a set of sampled D servers and this queue is "robbed" of half of its jobs which will then be added to the queue that had just emptied.

4 What is the answer to the original questions you asked?

The answer to our question is: "It depends".
In fact, we did many tests with different algorithm for the dynamic-d, different numbers of servers and lambdas and we noticed that the first four algorithms are more efficient than the fifth (that is the original simulator) only if the number of servers is low and lambda is high, as we can see in the following plots:

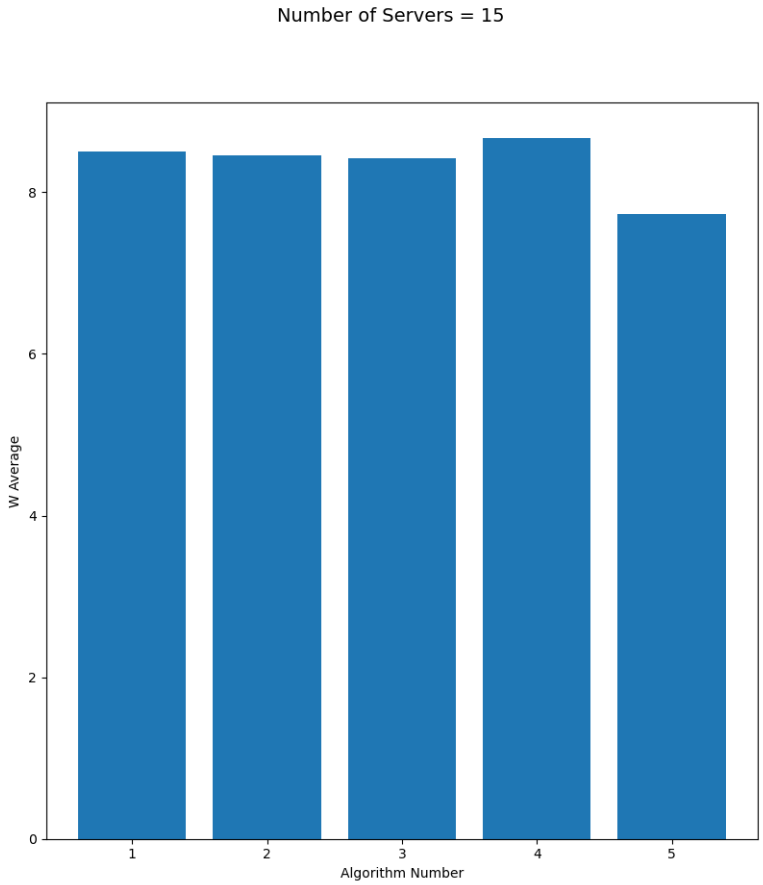


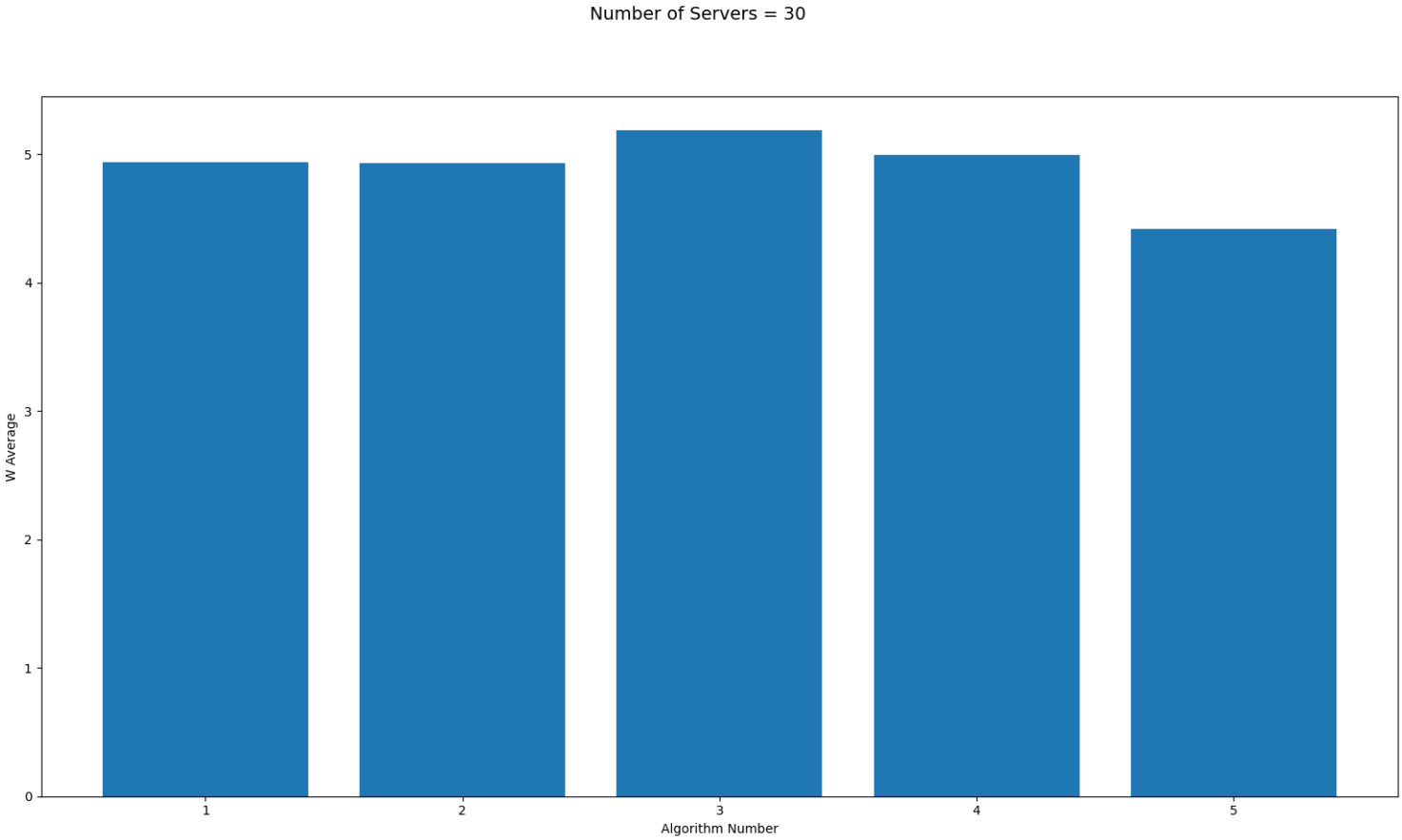
(a) lambda = 0.5

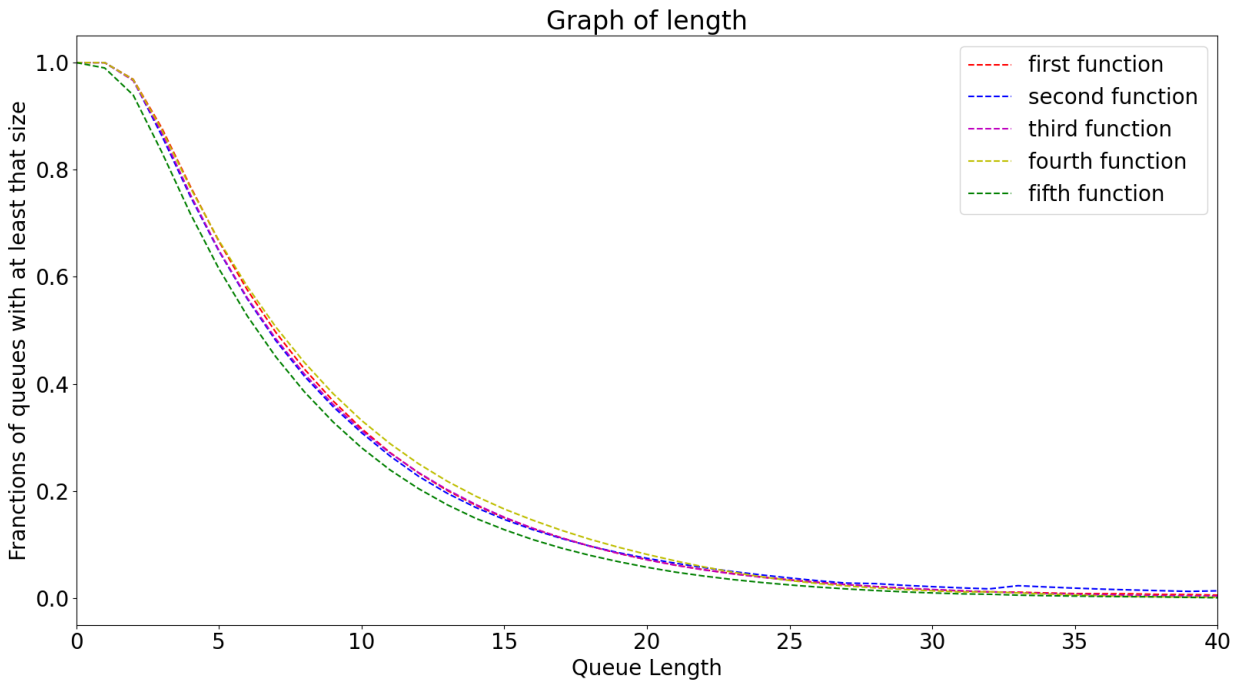


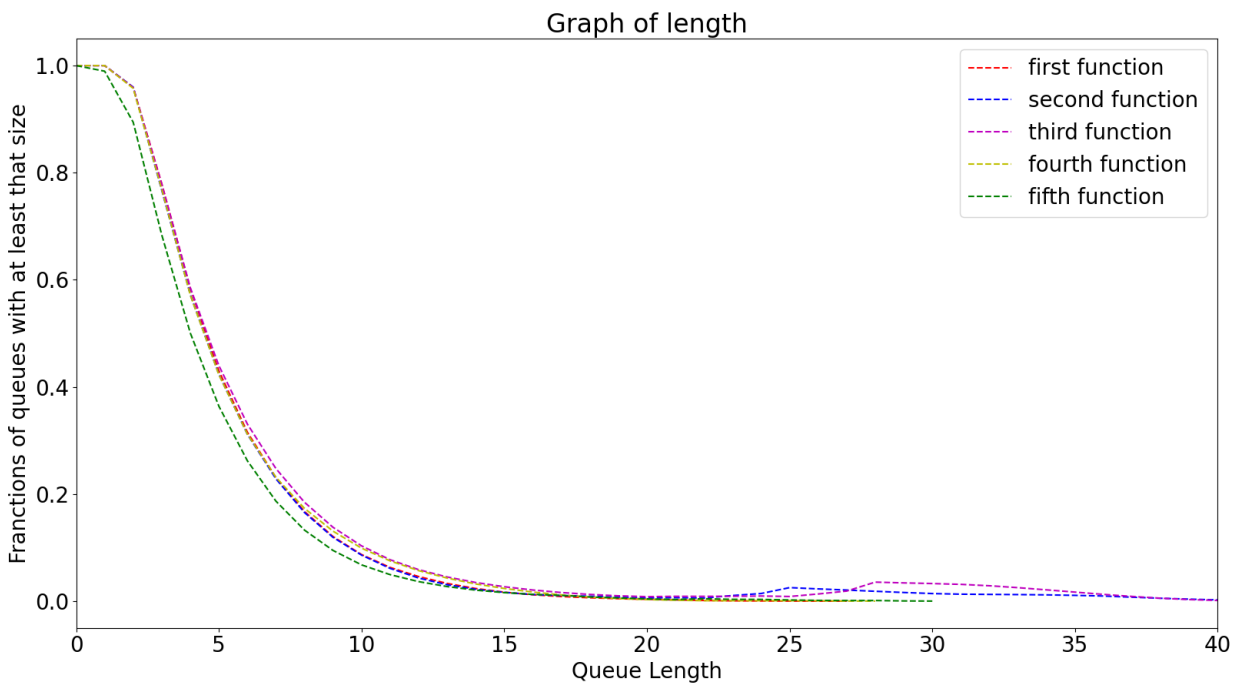
(b) lambda = 0.99

But it can be seen that by increasing the number of servers, the original system again performs better than the first four algorithms:


Number of Servers = 15
W Average
0
2
4
6
8
1
2
3
4
5
Algorithm Number


Number of Servers = 30
W Average
Algorithm Number
0
1
2
3
4
5
1
2
3
4
5


Graph of length
first function
second function
third function
fourth function
fifth function
Franctions of queues with at least that size
Queue Length
0.0
0.2
0.4
0.6
0.8
1.0
0
5
10
15
20
25
30
35
40


Graph of length
first function
second function
third function
fourth function
fifth function
Franctions of queues with at least that size
Queue Length
0.0
0.2
0.4
0.6
0.8
1.0
0
5
10
15
20
25
30
35
40

5 Why is that? What is your interpretation of the results you obtained?

As we expected, with a low lambda (0.5), the original system (the fifth algorithm, with d = N/2, where N is the number of servers) is the one that works best, since the queues, with a low workload, are mostly not very unbalanced and therefore the use of the combination of dynamic-d and job stealing is almost useless.

As soon as the workload is increased (lambda 0.99) our algorithms can be a better alternative since the queues start to be larger and increasingly unbalanced and therefore the combination of dynamic-d and job stealing positively influences the waiting times of the jobs.

Instead, by increasing the number of servers, we can notice again that the original system performs better than our algorithms because similarly to what happens with a low lambda, the average size of the queues decreases and therefore dynamic-d plus the job stealing turns out to be less and less useful as N increases.